Flow Algorithms

> → **Key Idea**: Model problems as networks with capacity constraints to find optimal flow

> • **When to use**: Maximum flow, minimum cut, bipartite matching, assignment problems

> ⊕ **Complexity**: $O(V^2E)$ for Ford-Fulkerson, $O(V^3)$ for maximum bipartite matching

> ⚠ **Common Pitfall**: Not handling residual graphs correctly or missing augmenting paths

Overview

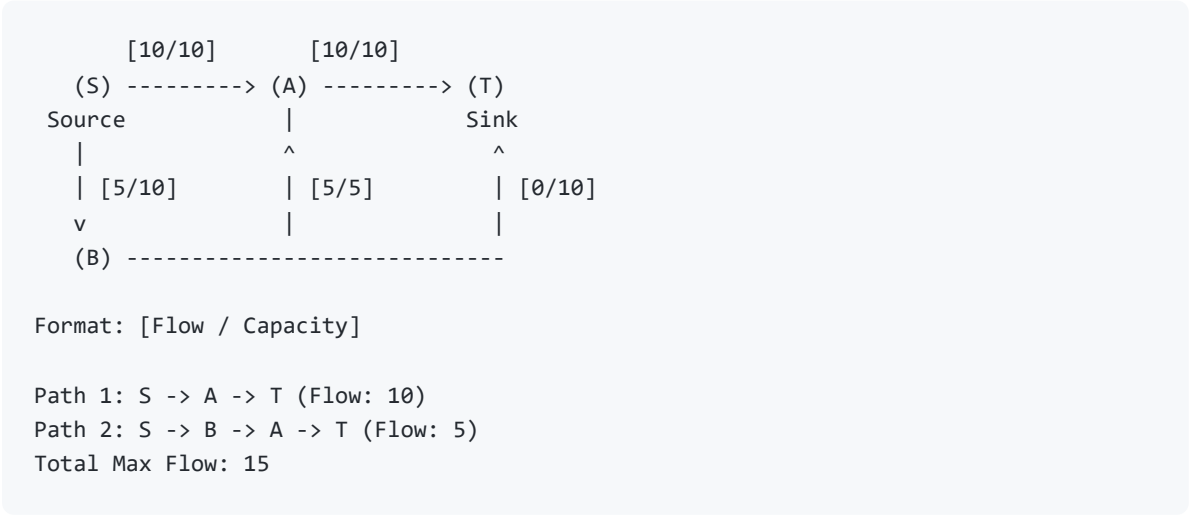
Flow algorithms solve optimization problems on networks by finding maximum flow from source to sink while respecting capacity constraints. Key applications include network routing, resource allocation, and matching problems.

Flow networks consist of:

- **Vertices**: Nodes in the network
- **Edges**: Connections with capacity constraints
- **Source**: Starting point of flow
- **Sink**: Destination point of flow
- **Capacity**: Maximum flow allowed through an edge

Visual Example

Maximum Flow Network



Core Implementation

Ford-Fulkerson Algorithm (DFS-based)

```
import java.util.*;

public class MaxFlow {
    static class Edge {
        int to, capacity, flow;

        Edge(int to, int capacity) {
            this.to = to;
            this.capacity = capacity;
            this.flow = 0;
        }

        int residualCapacity() {
            return capacity - flow;
        }
    }

    private List<List<Edge>> graph;
    private boolean[] visited;

    public MaxFlow(int n) {
        graph = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            graph.add(new ArrayList<>());
        }
        visited = new boolean[n];
    }

    public void addEdge(int from, int to, int capacity) {
        graph.get(from).add(new Edge(to, capacity));
        graph.get(to).add(new Edge(from, 0)); // Reverse edge for residual graph
    }

    /**
     * Ford-Fulkerson algorithm using DFS
     */
    public int maxFlow(int source, int sink) {
        int totalFlow = 0;

        while (true) {
            Arrays.fill(visited, false);
            int pathFlow = dfs(source, sink, Integer.MAX_VALUE);

            if (pathFlow == 0) break; // No augmenting path found

            totalFlow += pathFlow;
        }

        return totalFlow;
```

```
    }

    private int dfs(int node, int sink, int minCapacity) {
        if (node == sink) return minCapacity;

        visited[node] = true;

        for (Edge edge : graph.get(node)) {
            if (!visited[edge.to] && edge.residualCapacity() > 0) {
                int pathFlow = dfs(edge.to, sink,
                    Math.min(minCapacity, edge.residualCapacity()));

                if (pathFlow > 0) {
                    edge.flow += pathFlow;

                    // Update reverse edge
                    for (Edge reverseEdge : graph.get(edge.to)) {
                        if (reverseEdge.to == node) {
                            reverseEdge.flow -= pathFlow;
                            break;
                        }
                    }

                    return pathFlow;
                }
            }
        }

        return 0;
    }
}
```

Edmonds-Karp Algorithm (BFS-based)

```
public class EdmondsKarp {
    private int[][] capacity;
    private int[][] flow;
    private int[] parent;
    private int n;

    public EdmondsKarp(int n) {
        this.n = n;
        capacity = new int[n][n];
        flow = new int[n][n];
        parent = new int[n];
    }

    public void addEdge(int from, int to, int cap) {
        capacity[from][to] = cap;
    }

    /**
```

```
     * Edmonds-Karp algorithm using BFS
     */
    public int maxFlow(int source, int sink) {
        int totalFlow = 0;

        while (bfs(source, sink)) {
            int pathFlow = Integer.MAX_VALUE;

            // Find minimum capacity along the path
            for (int v = sink; v != source; v = parent[v]) {
                int u = parent[v];
                pathFlow = Math.min(pathFlow, capacity[u][v] - flow[u][v]);
            }

            // Update flow along the path
            for (int v = sink; v != source; v = parent[v]) {
                int u = parent[v];
                flow[u][v] += pathFlow;
                flow[v][u] -= pathFlow; // Reverse edge
            }

            totalFlow += pathFlow;
        }

        return totalFlow;
    }

    private boolean bfs(int source, int sink) {
        boolean[] visited = new boolean[n];
        Queue<Integer> queue = new LinkedList<>();

        queue.offer(source);
        visited[source] = true;
        parent[source] = -1;

        while (!queue.isEmpty()) {
            int u = queue.poll();

            for (int v = 0; v < n; v++) {
                if (!visited[v] && capacity[u][v] > flow[u][v]) {
                    queue.offer(v);
                    visited[v] = true;
                    parent[v] = u;
                    if (v == sink) return true;
                }
            }
        }

        return false;
    }
}
```

Key Optimizations

1. Residual Graph Construction

```
// Efficient residual graph using edge list
static class EdgeList {
    List<Edge> edges;
    List<List<Integer>> graph;

    void addEdge(int from, int to, int capacity) {
        graph.get(from).add(edges.size());
        edges.add(new Edge(to, capacity));

        graph.get(to).add(edges.size());
        edges.add(new Edge(from, 0)); // Reverse edge
    }

    int residualCapacity(int edgeId) {
        return edges.get(edgeId).capacity - edges.get(edgeId).flow;
    }
}
```

2. Level-based BFS (Dinic's Algorithm)

```
public class Dinic {
    private int[] level;
    private int[] iter;

    private boolean bfs(int source, int sink) {
        Arrays.fill(level, -1);
        level[source] = 0;

        Queue<Integer> queue = new LinkedList<>();
        queue.offer(source);

        while (!queue.isEmpty()) {
            int u = queue.poll();

            for (int edgeId : graph.get(u)) {
                Edge edge = edges.get(edgeId);

                if (level[edge.to] == -1 && edge.residualCapacity() > 0) {
                    level[edge.to] = level[u] + 1;
                    queue.offer(edge.to);
                }
            }
        }

        return level[sink] != -1;
    }
}
```

Common Problem Patterns

1. Bipartite Matching

Problem: Given a bipartite graph, find the maximum matching.

```
public int maxBipartiteMatching(int[][] graph) {
    int n = graph.length;
    int m = graph[0].length;

    MaxFlow flow = new MaxFlow(n + m + 2);
    int source = n + m;
    int sink = n + m + 1;

    // Connect source to left nodes
    for (int i = 0; i < n; i++) {
        flow.addEdge(source, i, 1);
    }

    // Connect right nodes to sink
    for (int j = 0; j < m; j++) {
        flow.addEdge(n + j, sink, 1);
    }

    // Connect left to right based on graph
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (graph[i][j] == 1) {
                flow.addEdge(i, n + j, 1);
            }
        }
    }

    return flow.maxFlow(source, sink);
}
```

2. Min-Cut Problem

Problem: Find the minimum cut that separates source from sink.

```
public List<List<Integer>> minCut(int source, int sink) {
    int maxFlowValue = maxFlow(source, sink);

    // Run BFS from source on residual graph
    boolean[] reachable = new boolean[n];
    Queue<Integer> queue = new LinkedList<>();
    queue.offer(source);
    reachable[source] = true;
```

```
        while (!queue.isEmpty()) {
            int u = queue.poll();

            for (Edge edge : graph.get(u)) {
                if (!reachable[edge.to] && edge.residualCapacity() > 0) {
                    reachable[edge.to] = true;
                    queue.offer(edge.to);
                }
            }
        }

        // Find cut edges
        List<List<Integer>> cutEdges = new ArrayList<>();
        for (int u = 0; u < n; u++) {
            if (reachable[u]) {
                for (Edge edge : graph.get(u)) {
                    if (!reachable[edge.to] && edge.capacity > 0) {
                        cutEdges.add(Arrays.asList(u, edge.to));
                    }
                }
            }
        }

        return cutEdges;
    }
```

3. Project Selection Problem

Problem: Given projects with profits and dependencies, maximize total profit.

```
    public int maxProfit(int[] profits, int[][] dependencies) {
        int n = profits.length;
        MaxFlow flow = new MaxFlow(n + 2);
        int source = n;
        int sink = n + 1;

        int totalPositive = 0;

        for (int i = 0; i < n; i++) {
            if (profits[i] > 0) {
                flow.addEdge(source, i, profits[i]);
                totalPositive += profits[i];
            } else {
                flow.addEdge(i, sink, -profits[i]);
            }
        }

        // Add dependency edges with infinite capacity
        for (int[] dep : dependencies) {
            int from = dep[0], to = dep[1];
            flow.addEdge(from, to, Integer.MAX_VALUE);
        }
```

```
        return totalPositive - flow.maxFlow(source, sink);
    }
```

4. Network Reliability

Problem: Find edge-connectivity of a network.

```
public int edgeConnectivity(int[][] graph) {
    int n = graph.length;
    int minCut = Integer.MAX_VALUE;

    // Try all pairs of vertices as source and sink
    for (int sink = 1; sink < n; sink++) {
        MaxFlow flow = new MaxFlow(n);

        // Add edges from adjacency matrix
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (graph[i][j] > 0) {
                    flow.addEdge(i, j, graph[i][j]);
                }
            }
        }

        int cut = flow.maxFlow(0, sink);
        minCut = Math.min(minCut, cut);
    }

    return minCut;
}
```

Advanced Variations

Push-Relabel Algorithm

```
public class PushRelabel {
    private int[] height, excess;
    private List<List<Edge>> graph;
    private Queue<Integer> activeVertices;

    public int maxFlow(int source, int sink) {
        initialize(source);

        while (!activeVertices.isEmpty()) {
            int u = activeVertices.poll();
```

```
                if (u != source && u != sink && excess[u] > 0) {
                    if (!push(u)) {
                        relabel(u);
                        activeVertices.offer(u);
                    }
                }
            }

            return excess[sink];
        }

        private boolean push(int u) {
            for (Edge edge : graph.get(u)) {
                if (edge.residualCapacity() > 0 && height[u] == height[edge.to] + 1) {
                    int pushFlow = Math.min(excess[u], edge.residualCapacity());

                    edge.flow += pushFlow;
                    excess[u] -= pushFlow;
                    excess[edge.to] += pushFlow;

                    if (excess[edge.to] == pushFlow && edge.to != source) {
                        activeVertices.offer(edge.to);
                    }

                    return excess[u] == 0;
                }
            }
            return false;
        }

        private void relabel(int u) {
            int minHeight = Integer.MAX_VALUE;

            for (Edge edge : graph.get(u)) {
                if (edge.residualCapacity() > 0) {
                    minHeight = Math.min(minHeight, height[edge.to]);
                }
            }

            height[u] = minHeight + 1;
        }
    }
```

Interview Tips

Pattern Recognition

- **Maximum flow problems**: Network capacity, resource allocation
- **Minimum cut problems**: Bottleneck identification, network vulnerability
- **Bipartite matching**: Assignment problems, stable marriages
- **Project selection**: Profit maximization with constraints

Common Mistakes

- **Forgetting reverse edges**: Essential for residual graph construction
- **Wrong termination condition**: Must check for augmenting paths correctly
- **Capacity overflow**: Use appropriate data types for large capacities
- **Not handling disconnected components**: Verify graph connectivity

Implementation Variations

```
// Template for flow network construction
class FlowNetwork {
    void addBidirectionalEdge(int u, int v, int capacity) {
        addEdge(u, v, capacity);
        addEdge(v, u, capacity);
    }

    void addDirectionalEdge(int u, int v, int capacity) {
        addEdge(u, v, capacity);
        addEdge(v, u, 0); // Reverse edge with 0 capacity
    }
}

// Template for multiple sources/sinks
void addSuperSource(int[] sources, int superSource) {
    for (int source : sources) {
        addEdge(superSource, source, Integer.MAX_VALUE);
    }
}
```

Time & Space Complexity

Algorithm	Time Complexity	Space Complexity	Notes
Ford-Fulkerson	$O(E \times f)$	$O(V + E)$	f = max flow value
Edmonds-Karp	$O(V \times E^2)$	$O(V^2)$	Always polynomial
Dinic's	$O(V^2 \times E)$	$O(V + E)$	Better for unit capacities
Push-Relabel	$O(V^2 \times E)$	$O(V + E)$	Good for dense graphs

---

← Previous: Java Data Structures | Back to Main Guide | Next: Main Guide →